Autoregressive Models

Factor a joint distribution into ordered conditional predictions. The same principle generates recipes token by token and images pixel by pixel, with different architectures enforcing the conditioning order.

Original study notes based on David Foster, Second edition, 2023. Book pp. 129–166. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.

Choose ordering → Prepare shifted targets → Learn conditionals → Sample next element → Append and repeat

Detailed Notes

Autoregressive factorization

Source: book pp. 129–131 / PDF p. 157

$$p(x_{1:n}) = \prod_{t=1..n} p(x_t | x_{<t})$$

This factorization is general, not a claim that the true process is inherently sequential. Choose an ordering and build a model that cannot access future targets. Training can use real prefixes, whereas generation conditions on the model's own past predictions, allowing early errors to compound.

Autoregressive models offer tractable conditional likelihoods but often slow sampling because each new element depends on earlier generated elements.

Recipe data, tokenization, and training windows

Source: book pp. 132–137 / PDF p. 160

Clean and standardize text while preserving task-relevant punctuation and recipe boundaries. Tokenization can use words, characters, or subwords, trading vocabulary size against sequence length and unknown-token handling.

Construct input sequences and targets shifted by one token. Include start/end markers and pad/mask consistently. Avoid windows or random splits that leak near-identical recipe fragments across train and evaluation data. Fit the vocabulary on training data under the chosen protocol and record it with the checkpoint.

Embedding layer and LSTM architecture

Source: book pp. 138–143 / PDF p. 166

An embedding maps token IDs to learned dense vectors. An LSTM processes vectors with a hidden state and a cell state. Forget, input, and output gates regulate retained information and exposure of the memory.

> $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
> $h_t = o_t \odot \tanh(c_t)$

A dense vocabulary head predicts next-token probabilities. Gates make long-range learning more practical than a plain recurrent cell, but do not provide unlimited memory. Track whether a layer returns every timestep or only the final state.

Training and sampling the LSTM

Source: book pp. 144–148 / PDF p. 172

Train with next-token cross-entropy, commonly using the true previous tokens as context (teacher forcing). Evaluate held-out loss and inspect generated recipe structure, repetition, vocabulary, and coherence.

> Temperature-adjusted sampling: $p_T(\text{token}) \propto p(\text{token})^{1/T}$

Lower temperature concentrates choices; higher temperature increases variety and risk of incoherence. Stop at an end marker or a safe length cap. Fluency and recipe-like formatting do not prove ingredients or cooking instructions are valid.

Stacked RNNs, GRUs, and bidirectional cells

Source: book pp. 149–153 / PDF p. 177

Stacked recurrent layers pass sequence representations upward; intermediate layers normally return sequences. GRUs use a simpler gated state update with fewer state mechanisms than LSTMs.

Bidirectional cells use context from both sequence directions and are useful when the whole input is available. A bidirectional layer must not inspect future target tokens in a causal generator. More layers and recurrence can increase cost or overfitting; compare under the same tokenization, data split, and objective.

PixelCNN: masked convolutions and residual blocks

Source: book pp. 154–158 / PDF p. 182

Treat an image as an ordered sequence of pixel values. Masked convolutions remove connections to future positions. The first mask excludes the current target; later masks may include representations that already obey causality. Residual blocks help train deeper networks without violating that information boundary.

Teacher-forced training evaluates many pixel conditionals in parallel because the complete training image is available, but masking prevents cheating. Sampling still fills an initially unknown image in order. A masking bug can yield excellent training likelihood and useless generation.

PixelCNN analysis and mixture distributions

Source: book pp. 159–166 / PDF p. 187

Inspect gradual image formation and compare output likelihood with sample appearance. Quantizing pixels into categories gives a simple classification objective; richer output distributions can model intensities more efficiently.

Mixture distributions combine several component densities with learned mixing weights, means, and scales. Compute log mixture likelihood stably with log-sum-exp. PixelCNN++-style mixtures and library implementations extend the basic masked model. Output-distribution choices determine what uncertainty is represented, while the causal mask determines what information is allowed.

Revision Check

1. Why can training be parallel while sampling stays sequential?
2. What do the LSTM gates control?
3. When would bidirectionality leak a target?
4. How do PixelCNN mask types protect causality?
5. How does temperature change diversity?

Autoregressive success depends on a valid ordering, correct shifted targets, and strict prevention of future-information leakage.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.